Assignment 4: Self-Supervised Representation Learning for Time Series Classification

Frederik Appel Vardinghus-Nielsen

September 11, 2026

Todo list

Contents

1 Part A: Understand the methods and connect paper to code

1.1 The three learning ideas

1.1.1 T-Loss (Franceschi et al., NeurIPS 2019)

The general idea is that sub-series of a reference series (which may be a sub-series of a larger time series) is a positiv sample, which sub-series of another series are negative samples. They use a CNN with dilation for increased receptive field. The series are left padded with zero if they are too short for the dilation of the network. If the series are long, more time stamps have representations in the end which is fixed with a max pool.

- Positive/negative samples: a positive example x^{pos} is a sub-series of the reference series x^{ref}, which itself is randomly sampled in length and location. A sub-series of another time series is negative.
- Objective: triple loss with one anchor, one positive, and $K \geq 1$ negative samples x_k^{neg}. The objective is to have the distance between x^{pos} and x^{ref} be smaller than the sum of the distances between x^{ref} and x_k^{neg}.
- Fixed length problem: the convolutional network ends with a representation $Y \in \mathbb{R}^{160 \times L}$ which is max pooled into $z \in \mathbb{R}^{160}$ and thereafter transformed through $f(x) = Wz + b$ where $W \in \mathbb{R}^{320 \times 160}$. The final dot product are therefore always between vectors of length 320. If the time series chosen in the beginning are too short for the full CNN they are left zero padded.
- Classification: an SVM with a radial basis function is trained on the features with their labels now avalable. It is evaluated in a train/test split.

1.1.2 TS2Vec (Yue et al., AAAI 2022)

The general idea is to generate related views with masking and random overlapping segments of the representations. The hierarchical loss furthermore lets the encoder learn representations which take into account a spectrum of resolutions (granularity of the series).

- Positive/negative samples: two different views of the same time series instance (segment) are achieved through random cropping and timestamp masking. The original time series is cropped to produced two distinct, overlapping time series. After and input projection layer, these are both randomly masked at specific time stamps (single timestamp, all dimensions for multivariate TS) by setting values to zero.
- Objective: Dual loss, the sum of two distinct expressions:
  - Temporal contrastive loss which compares views of the same overlapping series at the same and different timestamps. Similarity at same timestamp between two different views is positive, self-similarity (same view, no masking) at different timestamps is negative.
  - Instance-wise contrastive loss compares to other time series. Positives are two views of the same time series at the same timestamp. Negatives are both view from another time series at the same time stamp.

  The loss is calculated recursively at larger and larger granularity with a dilation of 2. The first level compares every single timestamp with all others. The second max pools with kernel size 2 and this continues until the full series are used.

- Fixed length problem: only the overlapping timestamps are used after passing the views through the encoder. The loss loops over time stamps and gives a single loss for them all.
- Classification: they use instance-level representations (the whole segment of time series, not just the overlap). Instances are passed through the encoder and a max pooling is performed over all timestamps, giving a feature vector of dimension K, which is the number of representation dimensions in the last layer. SVM is trained on this K-dimensional vector for classification.

1.1.3 TF-C (Zhang et al., NeurIPS 2022)

They seek to enforce discrimination in different domains This encourages that the time domain is robust to augmentation, the frequency domain is robust to augmentation, and the time-frequency domain enforces temporal and spectral consistency.

- Positive/negative samples:
  - Time: (1) Encodings of a time series and its augmented (jittering, scaling, time-shifts, neighborhood segments) view are similar and (2) encodings of two different time series (no augmentations) are dissimilar.
  - Frequency: (1) Encodings of the Fourier transform of a time series and its agumented view are similar and (2) encodings of two different series (no augmentations) are dissimilar.
  - Time-frequency: (1) Time series and their Fourier transform must be similar after first encoding into respective domains and thereafter encoding into common time-frequency domain, (2), augmented views are dissimilar.
- Objective: Combined loss in time-, frequency-, and time-frequency domain.
- Fixed length problem: The training datasets all contain series of same length. When using the model on a tuning dataset and testing, the series are left zero padded.
- Classification: the full architecture of $z_i^T = R_T(G_T(x_i^T))$ and $z_i^F = R_F(G_F(_i^F))$ is used and concatenated into $[z_i^T, z_i^F]$ and this is passed through a 2-layer fully ocnnected network which projects into the number of classes needed. This classifier is jointly trained with the full pipeline on the target dataset as fine tuning.

1.2 Paper-to-code map

For each element: (i) closest paper location, (ii) inputs and outputs, (iii) immediate caller or call site, and (iv) role in training or representation extraction. Verified checkout commits: T-Loss `4aff592`, TS2Vec `b0088e1`, TF-C `9667582`.

1.2.1 T-Loss

`losses/triplet_loss.py: TripletLoss.forward`

(i) Eq. (1)

(ii) **Input:** tensor `(B, C, L)`, where `B` is the batch size, `C` is the number of channels, and `L` is the length of the time series (equal length case).
Output: Loss as a scalar.

(iii) `TimeSeriesEncoderClassifier.fit_encoder`

(iv) It is the loss which dictates which time series are supposed to have similar and dissimilar representations.

`networks/causal_cnn.py: CausalConvolutionBlock.forward`

(i) Figure 2(b) in the paper.

(ii) **Input:** tensor `(B, C, L)`, where `B` is the batch size, `C` is the number of channels, and `L` is the length of the time series (equal length case).
Output: tensor `(B, C, L)`, where `B` is the batch size, `C` is the number of channels, and `L` is the length of the time series (equal length case). This is the results after passing through the convolutional block.

(iii) `CausalCNN` where is is called for each layer in the full network.

(iv) It's the basic block which dictates the neural network achitecture and therefore the inductive bias of the representation learning.

`networks/causal_cnn.py: CausalCNNEncoder.forward`

(i) Section 4 describes the full architecture. Figure 2 illustrates the convolutional blocks and dilation which are repeated in the first part of the encoder.

(ii) **Input:** tensor `(B, C, L)`, where `B` is the batch size, `C` is the number of channels, and `L` is the length of the time series (equal length case).
Output: tensor `(B, C)`, where `B` is the batch size and `C` is the number of channels. The time dimension L is removed with max pooling squeezing the time dimension.

(iii) `TimeSeriesEncoderClassifier.encode`

(iv) It wraps the whole classification pipeline of the encoder and calls it on the passed batch.

`scikit_wrappers.py: TimeSeriesEncoderClassifier.fit_encoder`

(i) Section 4 on architecture and Section S2 on hyperparameters.

(ii) **Input:** Training set X. **Output:** The fitted encoder.

(iii) `TimeSeriesEncoderClassifier.fit`

(iv) It performs unsupervised/self-supervised fitting of the encoder.

`scikit_wrappers.py: TimeSeriesEncoderClassifier.encode`

Fill in.

Required specifics

- How `fit_encoder` selects the fixed-length or varying-length loss:
  Fill in.
- How `TripletLoss.forward` samples reference, positive and negative intervals:
  Fill in.
- How `CausalCNNEncoder` produces one fixed-length vector:
  Fill in.
- How to call `fit_encoder` without labels:
  Fill in.

1.2.2 TS2Vec

`ts2vec.py: TS2Vec.fit`

Fill in.

`ts2vec.py: TS2Vec._eval_with_pooling`

Fill in.

`ts2vec.py: TS2Vec.encode`

Fill in.

`models/losses.py: hierarchical_contrastive_loss`

Fill in.

`models/losses.py: instance_contrastive_loss`

Fill in.

`models/losses.py: temporal_contrastive_loss`

Fill in.

`models/encoder.py: TSEncoder.forward`

Fill in.

`models/encoder.py: generate_binomial_mask`

Fill in.

`models/encoder.py: generate_continuous_mask`

Fill in.

Required specifics

- Overlapping crops in `TS2Vec.fit`:

  Fill in.

- Where timestamp masking is applied:

  Fill in.

- Mapping of the two elementary losses and hierarchical pooling to Equations 1–3 and Algorithm 1:

  Fill in.

- The `TS2Vec.encode` option returning one vector per complete series:

  Fill in.

1.2.3 TF-C

`code/TFC/dataloader.py: Load_Dataset.__init__`

Fill in.

`code/TFC/dataloader.py: Load_Dataset.__getitem__`

Fill in.

`code/TFC/dataloader.py: data_generator`

Fill in.

`code/TFC/dataloader.py: the fft.fft(...).abs() operation`

Fill in.

`code/TFC/augmentations.py: DataTransform_TD, DataTransform_FD, remove_frequency, add_frequency`

Fill in.

`code/TFC/model.py: TFC.__init__, TFC.forward, target_classifier.forward`

Fill in.

`code/TFC/loss.py: NTXentLoss_poly.forward`

Fill in.

`code/TFC/trainer.py: Trainer, model_pretrain, model_finetune, model_test`

Fill in.

`code/TFC/main.py: the subset setting, call to data_generator, pretrained-checkpoint loading`

Fill in.

1.2.4 Following one TF-C batch through the pipeline

Fill in.

FFT dimension, spectrum conversion, `fft` vs. `rfft`

Fill in.

Active time-domain augmentation; the two frequency operations and how they are combined

Fill in.

Attributes defining the time encoder, frequency encoder and the two projectors; the four outputs of `TFC.forward` in terms of h and z

Fill in.

Loss terms in `model_pretrain` vs. Equations 1–4

Fill in.

Tensors concatenated for the downstream classifier; is the encoder frozen or fine-tuned?

Fill in.

`main.py` → `data_generator` → `Trainer` **for FD-A → FD-B** Number of source and target-training samples actually used, whether `FD_B/val.pt` is loaded, how often the test loader is evaluated, and whether test results influence the reported best result. Discrepancies from the paper's stated protocol are recorded, not repaired.

Fill in.

Device-specific tensor construction in `remove_frequency` and `add_frequency`

Fill in.

2 Part B: Raw-signal baselines

2.1 Protocol

Datasets: FordA, ArticularyWordRecognition, Epilepsy. Classifiers: logistic regression and RBF-kernel SVM on the vectorised raw signal. Label regimes: 100% and a stratified 10% subset. Three fixed seeds; the 10% subset of a given seed is reused by every method. Hyperparameters selected on training data only.

Fill in.

2.2 Results

Dataset	Model	Labels	Accuracy	Macro-F1
FordA	Logistic regression	100%		
FordA	Logistic regression	10%		
FordA	RBF-SVM	100%		
FordA	RBF-SVM	10%		
AWR	Logistic regression	100%		
AWR	Logistic regression	10%		
AWR	RBF-SVM	100%		
AWR	RBF-SVM	10%		
Epilepsy	Logistic regression	100%		
Epilepsy	Logistic regression	10%		
Epilepsy	RBF-SVM	100%		
Epilepsy	RBF-SVM	10%		

Table 1: Raw-signal baselines. Test accuracy and macro-F1 as mean ± standard deviation over three paired seeds.

2.3 Comments

Fill in.

3 Part C: Pretrain and evaluate T-Loss and TS2Vec

3.1 Protocol

Per dataset and seed: pretrain on the unlabelled official training series, extract one fixed-length representation per series, freeze the encoder, fit the probe on the 10% and 100% labelled training representations, and evaluate once on the official test set.

Fill in.

Principal representation test The probe treated as the principal representation test is:

Fill in.

3.2 Results

Dataset	Method	Probe	Labels	Accuracy	Macro-F1
FordA	T-Loss	Logistic regression	100%		
FordA	T-Loss	Logistic regression	10%		
FordA	T-Loss	RBF-SVM	100%		
FordA	T-Loss	RBF-SVM	10%		
FordA	TS2Vec	Logistic regression	100%		
FordA	TS2Vec	Logistic regression	10%		
FordA	TS2Vec	RBF-SVM	100%		
FordA	TS2Vec	RBF-SVM	10%		
AWR	T-Loss	Logistic regression	100%		
AWR	T-Loss	Logistic regression	10%		
AWR	T-Loss	RBF-SVM	100%		
AWR	T-Loss	RBF-SVM	10%		
AWR	TS2Vec	Logistic regression	100%		
AWR	TS2Vec	Logistic regression	10%		
AWR	TS2Vec	RBF-SVM	100%		
AWR	TS2Vec	RBF-SVM	10%		
Epilepsy	T-Loss	Logistic regression	100%		
Epilepsy	T-Loss	Logistic regression	10%		
Epilepsy	T-Loss	RBF-SVM	100%		
Epilepsy	T-Loss	RBF-SVM	10%		
Epilepsy	TS2Vec	Logistic regression	100%		
Epilepsy	TS2Vec	Logistic regression	10%		
Epilepsy	TS2Vec	RBF-SVM	100%		
Epilepsy	TS2Vec	RBF-SVM	10%		

Table 2: Frozen-representation classification. Mean ± standard deviation over three paired seeds.

3.3 Comparison with the raw-signal baselines

Fill in.

Dataset	Method	Pretraining time	Representation dimension
FordA	T-Loss		
FordA	TS2Vec		
AWR	T-Loss		
AWR	TS2Vec		
Epilepsy	T-Loss		
Epilepsy	TS2Vec		

Table 3: Training time and representation dimensionality.

3.4 Training time and representation dimensionality

4 Part D: Robustness to contiguous sensor dropout

4.1 Corruption design

Dropout proportion, number of intervals and replacement convention, fixed before inspecting any result, plus a brief justification. The same corrupted inputs are applied to all methods.

Fill in.

4.2 Results (100%-label models)

		Accuracy		Macro-F1	
Dataset	Method	Clean	Corrupted (Δ)	Clean	Corrupted (Δ)
FordA	Logistic regression (raw)				
FordA	RBF-SVM (raw)				
FordA	T-Loss				
FordA	TS2Vec				
AWR	Logistic regression (raw)				
AWR	RBF-SVM (raw)				
AWR	T-Loss				
AWR	TS2Vec				
Epilepsy	Logistic regression (raw)				
Epilepsy	RBF-SVM (raw)				
Epilepsy	T-Loss				
Epilepsy	TS2Vec				

Table 4: Clean versus contiguous-dropout test performance for the 100%-label models.

4.3 Discussion

Whether any apparent robustness comes from the representation, the classifier or the preprocessing convention.

Fill in.

5 Part E: Focused TS2Vec continuous masking ablation

5.1 Setup

On FordA: standard (binomial) TS2Vec masking versus continuous masking, with encoder, training budget, downstream probe, label regime and the three seeds matched.

Fill in.

5.2 Results

	Accuracy		Macro-F1	
Masking	Clean	Corrupted (Δ)	Clean	Corrupted (Δ)
Binomial (default)				
Continuous				

Table 5: FordA: standard versus continuous TS2Vec masking, clean and contiguous-dropout test sets.

5.3 Interpretation

Does continuous masking change (i) clean performance, (ii) corrupted performance, and (iii) sensitivity to corruption relative to the model's own clean score?

Fill in.

6 Part F: TF-C for industrial bearing fault classification (FD-A → FD-B)

6.1 Execution record

Item	Value
Commands (pretraining)	
Commands (fine-tuning / testing)	
Environment (Python, PyTorch, OS)	
Hardware	
Seed	
Data actually consumed	
Training time	
Checkpoint used	

Table 6: TF-C FD-A → FD-B execution record.

6.2 Classification result

Run	Accuracy	Macro-F1
This run (FD-B test)		
TF-C supplement, Table 5		

Table 7: FD-B classification result, compared cautiously with Table 5 of the TF-C supplement. Exact numerical reproduction is not expected.

6.3 What was transferred, where labels entered, and encoder status

What was transferred from FD-A, where FD-B labels entered the pipeline, and whether the final encoder was frozen or updated.

Fill in.

6.4 Material differences between the paper's protocol and the supplied execution path

Subset use, loss composition, validation use and test-set access, based on the Part A inspection.

Fill in.

7 Part G: Synthesis

7.1 When SSL provides a useful representation and when raw nonlinear classification remains competitive

Fill in.

7.2 Whether the effect of SSL becomes clearer with only 10% labels

Fill in.

7.3 Stability of conclusions across the paired seeds

Fill in.

7.4 Role of dataset size, dimensionality and number of classes

Fill in.

7.5 Whether continuous masking improves accuracy, robustness, both or neither

Fill in.

7.6 What the TF-C bearing experiment shows about transfer between operating conditions

Fill in.

7.7 Implications for future multichannel industrial sensor data

Fill in.

8 Experimental rules: compliance notes

- Official test sets preserved as held-out data:

  Fill in.

- Test inputs not used for SSL pretraining in the required experiments:

  Fill in.

- Normalisation, imputation and feature transforms fitted on training data only:

  Fill in.

- 10% label subsets stratified, indices saved:

  Fill in.

- Same seeds, label subsets, corruptions and probe settings in paired comparisons:

  Fill in.

- Tuning on a training-only validation split or cross-validation:

  Fill in.

- Encoder frozen in Parts C–E; Part F follows the supplied TF-C downstream procedure:

  Fill in.

- Configurations, software versions, runtime and hardware recorded:

  Fill in.

- Implementation departures from the official repositories:

  Fill in.

9 Limitations and conclusions

Fill in.

A Optional extension: TF-C on FordA

Fill in.

B Optional extension: FordB noisy-condition experiments

Fill in.

C Per-seed results

Fill in.